```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_absolute_error, mean_squared_error,
r2_score
pd.set_option('display.width', 90)
pd.set_option('display.max_columns', None)
# ---------- 1. Create and load the vehicle data ----------
rng = np.random.RandomState(42)
n = 2000
cyl = rng.choice([3, 4, 6, 8], size=n, p=[0.03, 0.50, 0.30, 0.17])
disp = np.round(cyl * 50 + rng.normal(0, 20, size=n), 1)
hp = np.round(0.35 * disp + 55 + rng.normal(0, 12, size=n), 0)
wt = np.round(1200 + 3.2 * disp + 4.0 * hp + rng.normal(0, 150, size=n),
0)
acc = np.round(22 - 0.045 * hp + rng.normal(0, 1.2, size=n), 1)
year = rng.randint(70, 83, size=n)
origin = rng.choice([1, 2, 3], size=n, p=[0.60, 0.20, 0.20])
mpg = (44.0 - 0.005 * wt - 0.035 * hp + 0.25 * (year - 70)
       + 1.5 * (origin == 3) + 1.0 * (origin == 2)
       + rng.normal(0, 1.8, size=n))
df = pd.DataFrame({
    'Cylinders': cyl,
    'Displacement': disp,
    'Horsepower': hp,
    'Weight': wt,
    'Acceleration': acc,
    'Model_Year': year,
    'Origin': origin,
    'MPG': np.round(mpg, 1),
})
print(df.head())
print("Shape:", df.shape)
print("\nMissing values:\n", df.isnull().sum())
# ---------- 2. Univariate analysis ----------
```

```
def univariate_analysis(df, column):
    freq = df[column].value_counts()
    mean = df[column].mean()
    median = df[column].median()
    mode = df[column].mode()[0]
    std = df[column].std()
    return freq, mean, median, mode, std
freq, mean, median, mode, std = univariate_analysis(df, 'MPG')
print("\nUnivariate Analysis for MPG")
print("Frequency (top 5):\n", freq.head())
print("Mean:", mean)
print("Median:", median)
print("Mode:", mode)
print("Standard Deviation:", std)
print("\nCorrelation with MPG:")
print(df.corr()['MPG'].drop('MPG').round(3))
plt.figure(figsize=(10, 4))
plt.hist(df['MPG'], bins=30, color='tab:blue')
plt.title("Distribution of MPG")
plt.xlabel("MPG")
plt.ylabel("Number of Vehicles")
plt.show()
plt.figure(figsize=(10, 4))
plt.scatter(df['Weight'], df['MPG'], s=6, color='tab:green')
plt.title("Weight vs MPG")
plt.xlabel("Weight")
plt.ylabel("MPG")
plt.show()
# ---------- 3. Features and target ----------
features = ['Cylinders', 'Displacement', 'Horsepower', 'Weight',
            'Acceleration', 'Model_Year', 'Origin']
X = df[features]
y = df['MPG']
# ---------- 4. Train-test split ----------
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print("\nTotal, Train, Test:", len(df), len(X_train), len(X_test))
scaler = StandardScaler()
X_train_sc = scaler.fit_transform(X_train)
X_test_sc = scaler.transform(X_test)
# ---------- 5. Linear regression ----------
lin_reg = LinearRegression()
lin_reg.fit(X_train_sc, y_train)
```

```
pred_lr = lin_reg.predict(X_test_sc)
print("\nLinear Regression")
print("Coefficients:", lin_reg.coef_)
print("Intercept:", lin_reg.intercept_)
print("R2 Score:", r2_score(y_test, pred_lr))
# ---------- 6. Decision tree ----------
tree = DecisionTreeRegressor(max_depth=6, random_state=42)
tree.fit(X_train, y_train)
pred_dt = tree.predict(X_test)
print("\nDecision Tree")
print("R2 Score:", r2_score(y_test, pred_dt))
# ---------- 7. Random forest ----------
forest = RandomForestRegressor(n_estimators=100, max_depth=8,
random_state=42)
forest.fit(X_train, y_train)
pred_rf = forest.predict(X_test)
print("\nRandom Forest")
print("R2 Score:", r2_score(y_test, pred_rf))
print("Feature Importance:")
for name, imp in sorted(zip(features, forest.feature_importances_),
key=lambda t: -t[1]):
    print(" ", name, ":", round(imp, 4))
# ---------- 8. Naive baseline ----------
naive_pred = np.full(len(y_test), y_train.mean())
print("\nNaive baseline (predict average MPG for all cars)")
print("Baseline MAE:", mean_absolute_error(y_test, naive_pred))
print("Baseline R2:", r2_score(y_test, naive_pred))
# ---------- 9. Comparison ----------
results = {}
for name, pred in [("Linear Regression", pred_lr),
                   ("Decision Tree", pred_dt),
                   ("Random Forest", pred_rf)]:
    results[name] = {
        "MAE": mean_absolute_error(y_test, pred),
        "MSE": mean_squared_error(y_test, pred),
        "RMSE": np.sqrt(mean_squared_error(y_test, pred)),
        "R2": r2_score(y_test, pred),
    }
results_df = pd.DataFrame(results).T
print("\nModel Comparison")
print(results_df.round(4))
best_name = results_df['R2'].idxmax()
print("Better model:", best_name)
```

```
# ---------- 10. Actual vs Predicted ----------
best_pred = {"Linear Regression": pred_lr, "Decision Tree": pred_dt,
             "Random Forest": pred_rf}[best_name]
plt.figure(figsize=(5, 4))
plt.scatter(y_test, best_pred, s=8, color='tab:blue')
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--
', label="Perfect prediction")
plt.title("Actual vs Predicted MPG - " + best_name)
plt.xlabel("Actual MPG")
plt.ylabel("Predicted MPG")
plt.legend()
plt.show()
# ---------- 11. R2 comparison ----------
plt.figure(figsize=(10, 4))
plt.bar(results_df.index, results_df['R2'], color=['tab:blue',
'tab:orange', 'tab:green'])
plt.title("R2 Score Comparison")
plt.xlabel("Model")
plt.ylabel("R2 Score")
plt.show()
# ---------- 12. Predict a new vehicle ----------
new_car = pd.DataFrame([[4, 140, 90, 2400, 15.5, 80, 3]],
columns=features)
print("\nPredicted MPG for new vehicle:", round(forest.predict(new_car)
[0], 2))
```